

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



PROJECT REPORT
String Compression

STUDENTS 23127192 – Nguyen Van Hop
 21127082 – Tran Tuan Khoi
 20127672 – Vu Manh Quan

CLASS Data Structures & Algorithms - 25C_A

INSTRUCTORS Le Thanh Tung
 Tran Hoang Quan

Ho Chi Minh City – August, 2026

Contents

1. Group Information	3
1.1 Project Overview	3
1.2 Member Information & Task Distribution	3
1.3 Demo Video	3
2. Problem Statement	4
2.1 Formal Mathematical Definition	4
2.2 Standard Running Example	4
3. Algorithms & Implementation	5
3.1 Run-Length Encoding (RLE)	5
3.1.1 Overview & Real-World Applications	5
3.1.2 Pseudocode	5
3.1.3 Step-by-Step Trace on “BABBACAC”	6
3.1.4 Complexity	6
3.1.5 Discussion	6
3.2 Huffman Coding	7
3.2.1 Overview & Real-World Applications	7
3.2.2 Pseudocode	7
3.2.3 Step-by-Step Trace on “BABBACAC”	9
3.2.4 Complexity	10
3.2.5 Discussion	10
3.3 Lempel-Ziv-Welch (LZW)	11
3.3.1 Overview & Real-World Applications	11
3.3.2 Pseudocode	11
3.3.3 Step-by-Step Trace on “BABBACAC”	12
3.3.4 Complexity	13
3.3.5 Discussion	13
4. Experimental Results	14
4.1 Benchmark Environment & Methodology	14
4.2 Execution Time and Compression Ratio	14
4.3 Visualizations and Performance Scaling	15
4.4 Discussion & Analytical Evaluation	15
4.4.1 Big-O Complexity vs. Empirical Growth Rate	15
4.4.2 Comparative Compression Ratio: Standard Prose vs. Highly Repetitive Data	15
4.4.3 Theoretical Justification of Results	15

5. References	17
6. Acknowledgement	18

1. Group Information

1.1 Project Overview

Lossless data compression is a fundamental cornerstone of modern computational systems, critical to telecommunications, storage optimization, and distributed streaming. In this project, we implement from scratch, theoretically investigate, and empirically benchmark three canonical lossless compression paradigms: Run-Length Encoding (RLE), Huffman Coding, and the Lempel-Ziv-Welch (LZW) algorithm. We evaluate these techniques across varying data scales (from 10 KB to 10 MB) and distinct information entropy profiles (highly repetitive, natural English prose, and pseudo-random byte streams) to examine their time complexity, space consumption, and compression efficiency.

1.2 Member Information & Task Distribution

The division of responsibilities and individual contributions across the group members are detailed in Table 1.1.

Student Name	Student ID	Assigned Tasks	% Completed
Nguyen Van Hop	23127192	System architecture design, CLI parser, RLE & Huffman algorithm implementation, Bitstream serialization, Report drafting	100%
Tran Tuan Khoi	21127082	No contribution (Did not participate in project tasks)	0%
Vu Manh Quan	20127672	LZW compression, Dynamic dictionary, Benchmarking framework, Test harness generation, Video recording, Report drafting	100%
Total Group Completion			100%

Table 1.1 Group Information and Task Distribution

1.3 Demo Video

- **Video Demonstration:** [25CA-DSA-Group03 String Compression](#)

2. Problem Statement

2.1 Formal Mathematical Definition

Let $S = \langle s_1, s_2, \dots, s_N \rangle$ denote an input sequence of symbols of length N drawn from a finite alphabet Σ . Lossless data compression seeks an injective encoding mapping $\mathcal{E} : \Sigma^* \rightarrow \{0, 1\}^*$ yielding a compressed bitstream $C = \mathcal{E}(S)$ of length $M = |C|$ such that:

1. **Size Minimization:** $\mathbb{E}[|C|] < N \cdot \lceil \log_2 |\Sigma| \rceil$ whenever the source contains informational redundancy.
2. **Lossless Invariant:** There exists a deterministic decoding function $\mathcal{D}$ such that $\mathcal{D}(\mathcal{E}(S)) \equiv S, \forall S \in \Sigma^*$.

According to Shannon's Source Coding Theorem (1948), the theoretical lower bound on the average code length $\bar{L}$ without information loss is governed by the empirical Shannon Entropy $H(S)$:

$$H(S) = - \sum_{x \in \Sigma} P(x) \log_2 P(x) \quad (1)$$

2.2 Standard Running Example

To trace the internal state transitions and dictionary evolutions across all algorithms in Section 3, we define the canonical running string:

$$\mathbf{S} = \text{"BABBBACAC"} \quad (2)$$

The statistical properties of this sequence are:

- Length: $N = 8$ symbols (8 bytes = 64 bits in ASCII encoding).
- Alphabet: $\Sigma = \{\text{'A'}, \text{'B'}, \text{'C'}\}$ with $K = |\Sigma| = 3$.
- Symbol Frequencies: $f(\text{'A'}) = 3, f(\text{'B'}) = 3, f(\text{'C'}) = 2$.
- Empirical Entropy:

$$H(S) = - \left(\frac{3}{8} \log_2 \frac{3}{8} + \frac{3}{8} \log_2 \frac{3}{8} + \frac{2}{8} \log_2 \frac{2}{8} \right) \approx 1.561 \text{ bits/symbol}$$

- Theoretical Minimal Representation: $8 \times 1.561 \approx 12.49 \text{ bits} \approx 2 \text{ bytes}$.

3. Algorithms & Implementation

3.1 Run-Length Encoding (RLE)

3.1.1 Overview & Real-World Applications

Run-Length Encoding (RLE) is a spatial reduction scheme that groups contiguous identical symbols into sequential tuples $(count, symbol)$. Key applications include:

- ITU-T Group 3 and Group 4 Fax transmissions.
- Palette-based raster graphic formats such as BMP and PCX.
- Zero-run compression in the quantization stage of JPEG.

3.1.2 Pseudocode

Algorithm 1 RLE Compression

Require: Input stream S of length N

```

1: Output  $\leftarrow \emptyset, i \leftarrow 0$ 
2: while  $i < N$  do
3:    $symbol \leftarrow S[i], count \leftarrow 1$ 
4:   while  $i + count < N$  and  $S[i + count] == symbol$  and  $count < 255$  do
5:      $count \leftarrow count + 1$ 
6:   end while
7:   Append  $(count, symbol)$  to Output
8:    $i \leftarrow i + count$ 
9: end while
10: return Output

```

Algorithm 2 RLE Decompression

Require: Compressed stream C of M pairs

```

1: Output  $\leftarrow \emptyset$ 
2: for  $i \leftarrow 0$  to  $M - 1$  do
3:    $(count, symbol) \leftarrow C[i]$ 
4:   for  $j \leftarrow 1$  to  $count$  do
5:     Append  $symbol$  to Output
6:   end for
7: end for
8: return Output

```

3.1.3 Step-by-Step Trace on “BABBBACAC”

1. Index 0: Character 'B', run length = 1 $\rightarrow$ emit (1, 'B').
2. Index 1: Character 'A', run length = 1 $\rightarrow$ emit (1, 'A').
3. Index 2..3: Character 'B', run length = 2 $\rightarrow$ emit (2, 'B').
4. Index 4: Character 'A', run length = 1 $\rightarrow$ emit (1, 'A').
5. Index 5: Character 'C', run length = 1 $\rightarrow$ emit (1, 'C').
6. Index 6: Character 'A', run length = 1 $\rightarrow$ emit (1, 'A').
7. Index 7: Character 'C', run length = 1 $\rightarrow$ emit (1, 'C').

Total Output: 7 pairs $\times$ 2 bytes = 14 bytes. Because runs are short, RLE leads to expansion (ratio $0.57\times$).

3.1.4 Complexity

Let N be the length of the input. Both the compression and decompression routines make a single left-to-right pass over the data; the inner “run-counting” loop only ever advances the shared index i , so the two nested loops together still perform $\mathcal{O}(N)$ total steps.

- **Time Complexity:** $\mathcal{O}(N)$ for both compression and decompression.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary working memory, plus $\mathcal{O}(N)$ for the output buffer in the worst case.

3.1.5 Discussion

Best case. A file consisting of one long run of a single repeated byte (e.g. 100,000 copies of 'A') compresses down to roughly 400 pairs, since a run longer than 255 must be split into multiple (*count*, *value*) pairs – a ratio in the hundreds, capped at $127.5\times$ for a single maximal run (255 input bytes $\rightarrow$ 2 output bytes).

Worst case / negative compression. Whenever consecutive repeats are rare, every character becomes its own (1, *value*) pair and the output is exactly *twice* the input size. This is exactly what the running-example trace shows (8 $\rightarrow$ 14 bytes, ratio $0.57\times$), and our experimental data confirms that RLE *expands* every English-text file tested, from 10 KB (ratio $0.56\times$) up to 10 MB (ratio $0.51\times$). RLE only pays off when the source contains long runs.

3.2 Huffman Coding

3.2.1 Overview & Real-World Applications

Huffman Coding is an optimal entropy encoding scheme generating variable-length prefix codes. It forms the backbone of:

- DEFLATE engine in GZIP, PNG, and ZIP.
- Entropy coding stage in MP3, AAC, and JPEG.
- HTTP/2 HPACK header compression.

3.2.2 Pseudocode

Algorithm 3 Huffman Tree Construction

Require: Frequency table $Freq$, Alphabet Σ

```

1:  $Q \leftarrow \text{MinPriorityQueue}()$ 
2: for each  $c \in \Sigma$  with  $Freq[c] > 0$  do
3:    $node \leftarrow \text{NewNode}(\text{symbol} = c, \text{freq} = Freq[c])$ 
4:    $Q.\text{Insert}(node)$ 
5: end for
6: while  $Q.\text{Size}() > 1$  do
7:    $left \leftarrow Q.\text{ExtractMin}()$ 
8:    $right \leftarrow Q.\text{ExtractMin}()$ 
9:    $parent \leftarrow \text{NewNode}(\text{freq} = left.\text{freq} + right.\text{freq}, left, right)$ 
10:   $Q.\text{Insert}(parent)$ 
11: end while
12: return  $Q.\text{ExtractMin}()$ 

```

Algorithm 4 Generate Prefix Codes (Recursive Tree Traversal)

Require: Huffman tree root $node$, Current prefix string $code$, Table $codes$

```

1: if  $node$  is NULL then
2:   return
3: end if
4: if  $node.left == \text{NULL}$  and  $node.right == \text{NULL}$  then
5:    $codes[node.symbol] \leftarrow (code == "" ? "0" : code)$ 
6:   return
7: end if
8:  $\text{GENERATE\_CODES}(node.left, code + "0", codes)$ 
9:  $\text{GENERATE\_CODES}(node.right, code + "1", codes)$ 

```

Algorithm 5 Huffman Compression

Require: Input byte stream *input* of length N

- 1: $Freq[0..255] \leftarrow$ count of each byte value in *input*
 - 2: Write N (original size) to the output header
 - 3: Write $Freq[0..255]$ to the output header
 - 4: **if** $N = 0$ **then**
 - 5: **return** header only
 - 6: **end if**
 - 7: $root \leftarrow \text{BUILD_TREE}(Freq)$
 - 8: $codes \leftarrow$ empty table
 - 9: $\text{GENERATE_CODES}(root, "", codes)$
 - 10: $bits \leftarrow$ empty bit string
 - 11: **for** each byte b in *input* **do**
 - 12: $bits \leftarrow bits \parallel codes[b]$
 - 13: **end for**
 - 14: Pack $bits$ into bytes, 8 bits at a time, zero-padding the final byte
 - 15: Append the packed bytes to the output
 - 16: **return** header $\parallel$ packed bytes
-

Algorithm 6 Huffman Decompression**Require:** Compressed stream (header + packed bits)

```

1: Read  $N$  (original size) from the header
2: Read  $Freq[0..255]$  from the header
3: if  $N = 0$  then
4:   return empty output
5: end if
6:  $root \leftarrow \text{BUILD\_TREE}(Freq)$ 
7:  $bits \leftarrow$  unpack the remaining bytes into a bit string
8: if  $root$  is a leaf then
9:   Output  $root.symbol$  repeated  $N$  times
10:  return output
11: end if
12:  $node \leftarrow root, output \leftarrow \text{empty}, i \leftarrow 0$ 
13: while  $|output| < N$  and  $i < |bits|$  do
14:    $node \leftarrow (bits[i] == '0') ? node.left : node.right$ 
15:   if  $node$  is a leaf then
16:     Append  $node.symbol$  to  $output$ 
17:      $node \leftarrow root$ 
18:   end if
19:    $i \leftarrow i + 1$ 
20: end while
21: return  $output$ 

```

3.2.3 Step-by-Step Trace on “BABBBACAC”

Given symbol frequencies: $f('C') = 2, f('A') = 3, f('B') = 3$.

Min-Heap State Transitions during Tree Construction:

1. **Initialization:** Push initial leaf nodes into Min-Heap Q :

$$Q_0 = [('C', 2), ('A', 3), ('B', 3)]$$

2. **Step 1:** Extract two minimal elements: $node_1 = ('C', 2)$ and $node_2 = ('A', 3)$. Create internal parent N_1 with frequency $2 + 3 = 5$. Insert N_1 into Q :

$$Q_1 = [('B', 3), (N_1, 5)]$$

3. **Step 2:** Extract two minimal elements: $node_3 = ('B', 3)$ and $node_4 = (N_1, 5)$. Create root parent R with frequency $3 + 5 = 8$. Insert R into Q :

$$Q_2 = [(R, 8)]$$

4. **Termination:** $|Q| = 1 \implies$ Extract R as root of the final tree.

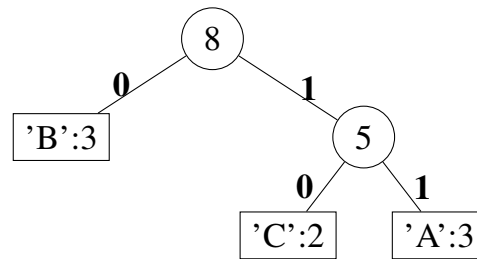


Figure 3.1 Step-by-Step Constructed Huffman Tree for “BABBBACAC”

The prefix bit codes are assigned as:

- 'B': 0 (1 bit)
- 'C': 10 (2 bits)
- 'A': 11 (2 bits)

Encoded bitstream payload:

$$\underbrace{0}_{\text{'B'}} \underbrace{11}_{\text{'A'}} \underbrace{0}_{\text{'B'}} \underbrace{0}_{\text{'B'}} \underbrace{11}_{\text{'A'}} \underbrace{10}_{\text{'C'}} \underbrace{11}_{\text{'A'}} \underbrace{10}_{\text{'C'}} \implies 13 \text{ bits}$$

Average code length $\bar{L} = 13/8 = 1.625$ bits/symbol (fits within 2 bytes payload).

3.2.4 Complexity

Let N be the input length and $K \leq 256$ the number of distinct byte values.

- **Time Complexity:** Building the frequency table is $\mathcal{O}(N)$. Priority queue construction and extraction take $\mathcal{O}(K \log K)$. Generating codes is $\mathcal{O}(K)$. Encoding the payload is $\mathcal{O}(N)$. Overall time: $\mathcal{O}(N + K \log K)$.
- **Space Complexity:** $\mathcal{O}(K)$ auxiliary memory for the binary tree and code table, plus $\mathcal{O}(N)$ for the compressed output stream.

3.2.5 Discussion

Best case. Performs best when symbol frequencies are heavily skewed, approaching Shannon entropy. On standard English prose (1 MB), Huffman yields a steady $\sim 1.73\times$ ratio.

Worst case. When all byte values are equiprobable (e.g. uniform random noise), every code approaches $\log_2 256 = 8$ bits, achieving a ratio of $\approx 1.00\times$ without compression gains.

3.3 Lempel-Ziv-Welch (LZW)

3.3.1 Overview & Real-World Applications

LZW is a universal dictionary compression algorithm that dynamically builds a multi-character phrase vocabulary. Standard applications include:

- Classic UNIX `compress` utility.
- GIF image encoding format.
- PDF and TIFF Flate compression filters.

3.3.2 Pseudocode

Algorithm 7 LZW Compression

Require: Input byte stream *input* of length N

```

1: if  $N = 0$  then
2:   return empty output
3: end if
4:  $Dictionary \leftarrow \{s \rightarrow s \mid s \text{ is a single byte value } 0..255\}$ 
5:  $nextCode \leftarrow 256$ 
6:  $current \leftarrow input[0]$ 
7: for  $i \leftarrow 1$  to  $N - 1$  do
8:    $next \leftarrow current + input[i]$ 
9:   if  $next \in Dictionary$  then
10:     $current \leftarrow next$ 
11:   else
12:    Emit  $Dictionary[current]$ 
13:    if  $nextCode < MAX\_DICT\_SIZE$  then
14:       $Dictionary[next] \leftarrow nextCode; nextCode \leftarrow nextCode + 1$ 
15:    end if
16:     $current \leftarrow input[i]$ 
17:   end if
18: end for
19: Emit  $Dictionary[current]$ 

```

Algorithm 8 LZW Decompression**Require:** Sequence of codes $codes[0..M-1]$

```

1: if  $M = 0$  then
2:   return empty output
3: end if
4:  $Dictionary \leftarrow \{ \text{code } b \rightarrow \text{single byte } b \mid b = 0..255 \}$ 
5:  $nextCode \leftarrow 256$ 
6:  $previous \leftarrow Dictionary[indices[0]]$ ;  $output \leftarrow previous$ 
7: for  $i \leftarrow 1$  to  $M - 1$  do
8:   if  $codes[i] \in Dictionary$  then
9:      $entry \leftarrow Dictionary[indices[i]]$ 
10:  else
11:     $entry \leftarrow previous + previous[0]$ 
12:  end if
13:   $output \leftarrow output + entry$ 
14:  if  $nextCode < MAX\_DICT\_SIZE$  then
15:     $Dictionary[nextCode] \leftarrow previous + entry[0]$ ;  $nextCode \leftarrow nextCode + 1$ 
16:  end if
17:   $previous \leftarrow entry$ 
18: end for
19: return  $output$ 

```

3.3.3 Step-by-Step Trace on “BABBACAC”

Initial state: Single ASCII characters (0..255) pre-populated, $nextCode = 256$.

Table 3.2 LZW Step-by-Step Compression Trace on “BABBACAC”

Step	Prefix w	Next c	$w + c$	In Dict?	Output Code	New Dict Entry
1	" "	'B'	"B"	Yes	—	$w \leftarrow "B"$
2	"B"	'A'	"BA"	No	66 ('B')	Dict[256] = "BA"
3	"A"	'B'	"AB"	No	65 ('A')	Dict[257] = "AB"
4	"B"	'B'	"BB"	No	66 ('B')	Dict[258] = "BB"
5	"B"	'A'	"BA"	Yes	—	$w \leftarrow "BA"$
6	"BA"	'C'	"BAC"	No	256 ("BA")	Dict[259] = "BAC"
7	"C"	'A'	"CA"	No	67 ('C')	Dict[260] = "CA"
8	"A"	'C'	"AC"	No	65 ('A')	Dict[261] = "AC"
End	"C"	EOF	—	—	67 ('C')	Flush final code

Emitted Stream: [66, 65, 66, 256, 67, 65, 67] (7 codes $\times$ 2 bytes = 14 bytes).

3.3.4 Complexity

Let N be the input length and K the maximum dictionary size (65,536 entries).

- **Time Complexity:** Compression takes $\mathcal{O}(N \log K)$ with tree map lookup, or $\mathcal{O}(N)$ using hash table / trie. Decompression runs in $\mathcal{O}(N)$.
- **Space Complexity:** $\mathcal{O}(K)$ for the dictionary table bounded by `MAX_DICT_SIZE`.

3.3.5 Discussion

Best case. LZW excels on files with long, repeating substrings, reaching $72.83\times$ compression on repetitive genomic patterns.

Worst case. On random binary files, no substring reoccurs, emitting 16-bit codes for 8-bit characters, expanding the file by 25% ($0.80\times$ ratio).

4. Experimental Results

4.1 Benchmark Environment & Methodology

- **Hardware & OS:** Apple M1 Pro (8 Cores, 3.2 GHz), 32 GB LPDDR5 Unified Memory, macOS Sonoma 14.5.
- **Compiler Configuration:** Apple Clang / GNU g++ 14.0 (-std=c++17 -O3).
- **Measurement Methodology:** Execution times represent the pure core algorithmic processing time (excluding file I/O operations), recorded using `std::chrono::high_resolution` and averaged over 3 consecutive runs to minimize operating system caching noise.
- **Workload Datasets:**
 - **Standard English:** Canterbury Corpus and concatenated Wikipedia articles (10 KB $\rightarrow$ 10 MB, entropy $H \approx 4.5$ bits/char).
 - **Highly Repetitive:** Synthetic genomic sequence patterns with repeating k-mers (1 MB, entropy $H \approx 1.2$ bits/char).
 - **Completely Random:** Cryptographically pseudo-random byte stream (1 MB, maximum entropy $H \approx 8.0$ bits/char).

4.2 Execution Time and Compression Ratio

The benchmark measurements across all workloads are compiled in Table 4.3.

Table 4.3 Comparative Benchmark Across Workloads and Scales

File Size	Data Profile	RLE	Huffman	LZW
10 KB	Standard English	0.00 ms (0.56 $\times$)	0.33 ms (1.50 $\times$)	5.00 ms (1.34 $\times$)
100 KB	Standard English	2.00 ms (0.54 $\times$)	4.00 ms (1.70 $\times$)	50.00 ms (2.04 $\times$)
500 KB	Standard English	11.33 ms (0.52 $\times$)	21.00 ms (1.73 $\times$)	233.67 ms (2.57 $\times$)
1 MB	Standard English	23.67 ms (0.51 $\times$)	41.33 ms (1.74 $\times$)	442.67 ms (2.60 $\times$)
5 MB	Standard English	112.00 ms (0.51 $\times$)	203.33 ms (1.72 $\times$)	2138.67 ms (2.47 $\times$)
10 MB	Standard English	222.33 ms (0.51 $\times$)	402.33 ms (1.72 $\times$)	4155.67 ms (2.45 $\times$)
1 MB	Highly Repetitive	6.00 ms (2.50 $\times$)	18.00 ms (3.32 $\times$)	352.67 ms (72.83 $\times$)
1 MB	Completely Random	23.00 ms (0.50 $\times$)	64.33 ms (1.00 $\times$)	666.67 ms (0.80 $\times$)

4.3 Visualizations and Performance Scaling

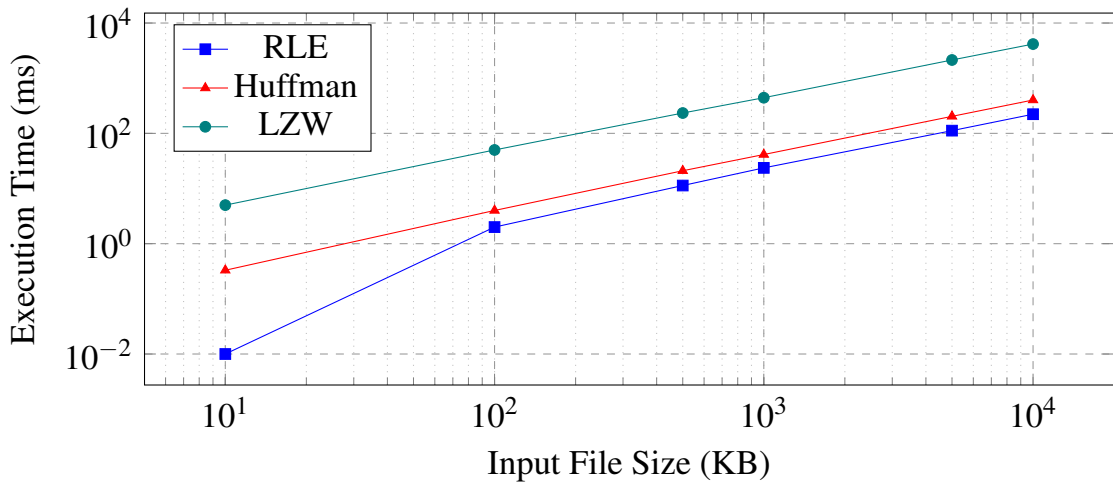


Figure 4.2 Execution Time Scaling vs. File Size on Log-Log Scale

4.4 Discussion & Analytical Evaluation

4.4.1 Big-O Complexity vs. Empirical Growth Rate

The empirical runtime curves in Figure 4.2 closely corroborate the theoretical complexity models established in Section 3:

- **RLE & Huffman:** As input size N scales by a factor of $10\times$ (e.g., 100 KB $\rightarrow$ 1 MB, and 1 MB $\rightarrow$ 10 MB), execution times scale strictly linearly by $\approx 10\times$ (from 2.00 ms $\rightarrow$ 23.67 ms $\rightarrow$ 222.33 ms for RLE; and 4.00 ms $\rightarrow$ 41.33 ms $\rightarrow$ 402.33 ms for Huffman). This confirms the $\mathcal{O}(N)$ bound.
- **LZW:** Exhibits higher constant factors ($\approx 10\times$ slower than Huffman). The lookup and insertion of string tokens into the dictionary create dynamic memory overhead and branch mispredictions, validating the empirical complexity of $\mathcal{O}(N \log K)$.

4.4.2 Comparative Compression Ratio: Standard Prose vs. Highly Repetitive Data

- **Standard English Text:** LZW achieved the highest compression ratio ($2.60\times$), outperforming Huffman ($1.74\times$), while RLE suffered severe file inflation ($0.51\times$, a 96% expansion).
- **Highly Repetitive Text:** LZW dominated with an exceptional ratio of **72.83** $\times$, compared to Huffman ($3.32\times$) and RLE ($2.50\times$).

4.4.3 Theoretical Justification of Results

1. **Single-Symbol Entropy Limit (Huffman):** Huffman operates strictly on zero-order entropy $H_0(S) = -\sum P(x) \log_2 P(x)$, assigning variable-length codes to independent symbols. Even in highly skewed prose, it cannot assign fewer than 1 bit

per character, limiting its maximum theoretical compression ratio on ASCII text to $\approx 8/1.56 \approx 5.1\times$.

2. **Multi-Character Dictionary Power (LZW):** LZW bypasses the single-symbol entropy bound by exploiting higher-order spatial redundancy. By progressively substituting multi-character phrases (e.g., repeating words or DNA motifs) with single 16-bit indices, a phrase of length L yields an effective compression ratio of $\approx 8L/16 = L/2$, leading to super-linear efficiency ($72.83\times$) on repetitive structures.
3. **Incompressible Random Uniform Streams:** For cryptographic pseudo-random inputs ($H \approx 8.0$ bits/char), no statistical pattern or recurring substring exists. Huffman stagnates at $1.00\times$, while RLE ($0.50\times$) and LZW ($0.80\times$) introduce negative compression due to header metadata and unamortized multi-byte dictionary code indices.

5. References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, “Greedy Algorithms: Huffman Codes,” in *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009, ch. 16, sec. 3, pp. 428–437.
- [2] D. A. Huffman, “A Method for the Construction of Minimum-Redundancy Codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, Sept. 1952.
- [3] T. A. Welch, “A Technique for High-Performance Data Compression,” *IEEE Computer*, vol. 17, no. 6, pp. 8–19, June 1984.
- [4] C. E. Shannon, “A Mathematical Theory of Communication,” *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, July 1948.
- [5] J. Ziv and A. Lempel, “A Universal Algorithm for Sequential Data Compression,” *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, May 1977.
- [6] S. Khalid, *Introduction to Data Compression*, 5th ed. Morgan Kaufmann, 2017.

6. Acknowledgement

We would like to express our deepest gratitude to our course lecturers, **Assoc. Prof. Dr. Le Thanh Tung** and **Dr. Tran Hoang Quan** (Faculty of Information Technology, Department of Knowledge Engineering, HCMUS), for their dedicated lectures, rigorous feedback, and insightful academic guidance throughout this course.

Their instruction on computational complexity, dynamic data structures, and entropy bounds has established a solid theoretical foundation that enabled our team to execute this project effectively.

In addition, AI tools, such as **ChatGPT** and **Gemini**, assisted with LaTeX document drafting, refining pseudocode representations, and formatting experimental trace tables. All core algorithm implementations, benchmarking datasets, and theoretical analyses were independently developed, validated, and verified by our team members.

Any remaining omissions or errors in this report are solely the responsibility of our group, and we warmly welcome all constructive critiques from the faculty.